

Mapping (Version 1.0)

```
Crypto_Verify(publicKey, message, signature) :=
    boolean ECDSAVerify(QU := publicKey, M := message, S := signature)
```

ECDSAVerify() SHALL be the ECDSA signature verification function as defined in Section 4.1.4 of [SEC 1](#) using **Crypto_Hash()** as the underlying hash function **Hash()**; returns **TRUE** if the verification succeeds and **FALSE** otherwise.

3.5.4. ECDH

Crypto_ECDH() is used to compute a shared secret from the Elliptic Curve Diffie-Hellman (ECDH) protocol.

Function and description

```
byte[CRYPTO_GROUP_SIZE_BYTES]
Crypto_ECDH(
    PrivateKey myPrivateKey,
    PublicKey theirPublicKey)
```

Computes a shared secret using Elliptic Curve Diffie-Hellman.

Mapping (Version 1.0)

```
Crypto_ECDH(myPrivateKey, theirPublicKey) :=
    byte[CRYPTO_GROUP_SIZE_BYTES] ECDH(dU := myPrivateKey, QV := theirPublicKey)
```

The output of **ECDH()** SHALL be the serialization of the x-coordinate of the resultant point as defined in Section 3.3.1 of [SEC 1](#).

3.5.5. Certificate validation

Crypto_VerifyChain() is used to verify [Matter certificates](#).

Crypto_VerifyChainDER() is used to verify public key [X.509 v3](#) certificates in [X.509 v3 DER](#) format.

Function and description

```
boolean
Crypto_VerifyChain(MatterCertificate[] certificates)
```

Given [Matter certificates](#), **Crypto_VerifyChain()** performs all the necessary validity checks on **certificates**, taking in account that the notion of "current time" for the purposes of validation SHALL abide by the rules in [Section 3.5.6, "Time and date considerations for certificate path validation"](#).